2 The C# object system

2.1 Object-oriented principles

Before we talk about the object system C#, it's good to reflect on the desired properties of object-oriented code (and how they are realised in the language): polymorphism, encapsulation, and extensibility.

Polymorphism deals with code operating over multiple types. Generics embody this principal. For example, the generic `List<T>` class from the .NET library is equivalent to `ArrayList<T>` from the Java class libraries. `List<T>` stores a list of objects of type T (where T is any type) in a (growable) array.

Another example is sorting a list. As long as the contained class defines some sort of comparison function between objects of its type, any reasonable sort function should work on it. In Java, such a class must implement the Comparable interface and in C#, they implement the `IComparable<T>` interface.

```
class IntPair : IComparable<IntPair>
{
    public int X { get; private set; }
    public int Y { get; private set; }
    public IntPair(int x, int y)
    {
        X = x;
        Y = y;
    }

    /// <summary>
    /// We compare pairs element-wise.
    /// </summary>
    public int CompareTo(IntPair other)
    {
        if (X < other.X)
        {
            return -1;
        }
        else if (X > other.X)
        {
            return 1;
```